Portfolio

Full-Stack AI Engineer

Bibek K C

kc-bibek.com.np · info@kc-bibek.com.np · github.com/bibekyess · linkedin.com/in/bibek44

Updated September 2026

SELECTED PROJECTS

KhururuAI: Work in Flow
Charak: Medical Research Assistant
Bintech International
AirConnect: Proximity-Based Social Discovery
B-Assist (Better-Assistant)
ReNameify: Content-Aware PDF Batch Renamer

RESEARCH AND APPLIED ML PROJECTS

FastRAG: Self-Hosted Document RAG Backend
Probabilistic Multiview Object-Pose Estimation
6D Pose Estimation and Scene Graph Generation
MBTI Prediction
ESL Fusion: Junction Asia Hackathon

EARLIER PROJECTS AND COURSEWORK

B-bot (Personal Chatbot)
Paraphrase Generation (QCPG Replication)
AI-empowered F1/10 Autonomous Racing Car
Crypto Price Predictor
Movie Recommendation (Netflix Prize style)
Pintos
Smart Home
Data Visualization Task

SELECTED PROJECTS

Products, designed and built end to end.

KhururuAI: Work in Flow WEB APP (2026)

The point of this project is access. Capable AI tools price out exactly the people who would gain most from them, while every major provider (Groq, Cerebras, NVIDIA, Google, Mistral) gives away a free tier that goes unclaimed because claiming it means holding an API key. KhururuAI pools that free capacity into a product anyone can just open and use: no payment, no API key, and no account at all for the first five turns. It runs today on my own pooled keys, roughly **2.3B tokens** of budget; the version I'm working toward provisions each user's *own* free tier at sign-in, so the ceiling scales with users instead of being divided among them.

A user writes what they want in plain language, an LLM router picks one of six specialists, and that specialist returns the finished thing: a researched answer with live citations, a slide deck, a PDF, working code, an image, or ordinary conversation. Uploaded documents are parsed, embedded and retrieved with page-accurate citations. Turns stream over SSE as typed events, so the interface names the specialist at ~1–2s and the model that is writing before the first token exists.

Staying free without falling over is the engineering problem. One OpenAI-compatible gateway fronts every provider, collapsing three vendor SDKs into a single client where "which provider" becomes "which model id"; agents are pinned to capability *slots* rather than vendors, so a model that runs dry is a one-line swap and an exhausted route falls back on its own.

Free access then sits behind two ceilings doing different jobs: a guest allowance that deliberately never refills, and a per-IP daily budget that stops the endpoint being farmed. Five Node services plus Redis, Qdrant and MinIO behind a React SPA, reproducible with one Docker Compose command, with the decisions recorded as ADRs.

> **Role:** Solo. Architecture, all five services, the SPA and deployment.
>
> **Tools:** Node.js, Express, LangGraph.js, React, TypeScript, MongoDB, Redis, Qdrant, MinIO/S3, Docker Compose, nginx, Firebase Auth, Tavily
>
> **ML/Algorithms:** LLM intent routing across an agent roster; multi-provider model routing with exhaustion fallback; document RAG (page-aware chunking, Gemini embeddings, score-floored retrieval under a hard context budget, validated citations)
>
> **Links:** khururu.com

Charak: Medical Research Assistant WEB APP MCP SERVER (2026)

For deterministic work, a tool-calling agent beats code-gen or one big prompt. Charak answers natural-language clinical questions from live PubMed literature and ClinicalTrials.gov v2 records, and charts the

answer when the question is really about a distribution. The LLM owns language and control flow only, orchestrating typed Python tools, and never sees or writes a number. State stays server-side and it passes only opaque dataset handles, so it cannot emit a wrong figure because it is never handed one. Chart-type selection is deterministic rules; the "no matching trials" reply is code-gated rather than left to judgment. That costs more round-trips, but each is a short tool call instead of a wall of records pushed through context — cheaper per query and structurally correct.

The backend renders nothing: it returns a small flat JSON spec that Vega-Lite draws on the client, thin enough that one response feeds the chat UI, the `POST /visualize` API and a standalone MCP server unchanged.

The hard part was the public API, not the model. I verified every capability with live `curl` first, which caught the wrong assumptions. The obvious filters (`filter.phase`, `filter.country`) don't exist; real narrowing is Essie, `AREA[Phase]PHASE3`. And the one aggregation endpoint can't be scoped to a query and returns no trial IDs, so it can't back citations either.

With no server-side aggregation, counts are resolved in tiers: exact from the full corpus, exact via a cheap per-bucket `countTotal` query when it's too large to fetch, sampled-but-flagged only for unbounded fields. Because aggregation walks records instead of asking for a summary, **every bar carries the trial IDs that built it**.

> **Role:** Solo. API research, agent and tool design, backend and frontend.
>
> **Tools:** Python, FastAPI, Pydantic v2, MCP (Model Context Protocol), Vega-Lite, vis-network, Server-Sent Events, Docker Compose, Caddy, pytest, uv
>
> **ML/Algorithms:** Tool-calling agent loop with server-side state handles (no numeric values in context); three-tier exact-count resolution over a non-aggregating API; deterministic data-shape → chart-type selection; entity-normalized co-occurrence network construction; provider-swappable LLM layer (OpenAI / Anthropic)
>
> **Links:** charak.khururu.com · GitHub

Bintech International WEB APP (2026)

Built to survive being handed over. Bintech is a B2B hardware supplier in Nepal. I shipped the whole thing solo: public site, staff CRM, AI assistant, and the server it runs on. Nobody there writes code, so the design is only correct if a salesperson can add a product, publish it and answer a customer without me. That put the catalogue in Payload CMS 3 over Postgres with real access control (anonymous reads constrained *by query* to published, staff see drafts) and admin copy written for the person typing into it. Inquiries invert the usual permissions: the public creates but can't read, staff read but the submitted fields are read-only, so the only editable thing is the New → Contacted → Closed status.

Failure modes took most of the attention. Every catalogue query falls back to a static array, so the image builds with no database and a Postgres outage degrades to the seed catalogue instead of a blank page; inquiry emails fire from a post-persist hook, so an SMTP outage costs a notification, never a lead.

The assistant is deliberately the boring kind. Each request flattens the live catalogue into one compact line per product behind a 60s cache and pins it into a guardrailed system prompt that forbids inventing prices and hard-codes the only two legal URL shapes (LLMs cheerfully guess `/products/monitors`, which 404s). Answers stream back as plain-text deltas; the provider key never reaches the browser, requests are per-IP rate-limited and length-capped, and each completion logs one structured line so a truncated answer is diagnosable from logs rather than guessed at.

Grounding beats fine-tuning here. A price edit in the CRM is live in the bot's answers within a cache window, and it cannot quote a number that isn't in the database. It self-hosts on one VPS via Docker Compose behind Caddy, deployed by a GitHub Actions job holding no repo permissions against an SSH key locked server-side to a forced command. Domain, DNS and company mailboxes are mine too.

> **Role:** Solo. Site, CRM, assistant, VPS, domain and DNS.
>
> **Tools:** Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4, Payload CMS 3, PostgreSQL 17, Drizzle migrations, Nodemailer/SMTP, sharp, Motion, Docker Compose, Caddy, GitHub Actions
>
> **ML/Algorithms:** Catalogue-grounded LLM assistant. Live DB → compact token-efficient context behind a TTL cache, guardrailed system prompt (no invented prices/specs, closed link vocabulary, out-of-scope hand-off), SSE → plain-text delta re-emission, per-IP rate limiting and per-completion telemetry
>
> **Links:** bintechint.com

AirConnect: Proximity-Based Social Discovery MOBILE APP (2025)

Ultra-Wideband was the bet, and the hardware wasn't ready. The idea was mine: you are on a subway and the person across from you is reading a book you love, and there is no socially cheap way to say so. AirConnect was the icebreaker: ping the one person in your line of sight, so the first move costs nothing.

That premise needs direction, not distance. UWB is the only widely shipped radio that gives angle of arrival rather than just range, and "that person, over there" is an angle problem. Bluetooth knows someone is nearby, never which one. The UWB SDKs proved too unstable to build on, and without direction the premise was gone. We pivoted to GPS-based nearby discovery, which put us in dating-app territory, where Apple now rejects new entrants that aren't meaningfully differentiated. So we stopped. **Two people, six weeks**: a working real-time app, and a clear answer to why the idea doesn't survive current hardware and platform policy.

An Expo/React Native client and a Node + Socket.IO backend: nearby discovery, matching, chat, and live location sharing on a map, with Redis as the primary datastore. Binh built the foundation and owned the map/GPS and chat features. **I owned the backend and infrastructure**: Docker/Compose, then restructuring a 2,500-line single-file Express server into controllers, services and routes with a PostgreSQL migration path alongside Redis, plus the welcome/signup/onboarding flow and a polish pass over Binh's map (marker-flicker fix, `DirectionMarker` and compass heading).

> **Role:** Two-person team. I owned the backend, infrastructure and the onboarding flow; Binh owned the map/GPS and chat.

> **Tools:** Expo, React Native, TypeScript, Node.js, Express, Socket.IO, Redis, PostgreSQL, Docker Compose, cloudflared

---

B-Assist (Better-Assistant) DESKTOP APP (2024)

The assistant should come to your text, not your text to the assistant. Every AI writing tool I used meant the same round-trip: leave the editor, paste into ChatGPT, read the answer, copy it back, paste again. B-Assist collapses that to a keystroke. Select text anywhere (an email, a Word document, an IDE), press `ctrl+shift+X` (`cmd+shift+X` on macOS), pick an action, and the processed text replaces your selection in place. You never leave the window you were writing in.

Working in *every* editor rules out integrating with any of them, so it sits under the apps rather than inside them: a global hotkey opens a small window, and robotJS drives the OS clipboard and keystrokes to lift the selection and type the result back — the same path whether the target is Gmail, Word or VS Code, with no per-app plugin to write or maintain. Models run locally through Ollama, which does two jobs at once: there is no API key to hand a non-technical user, and drafts being rewritten never leave the machine. It ships as a one-click executable for Windows and macOS, with grammar and spelling correction and English→Korean translation as the actions implemented.

> **Role:** Solo.
>
> **Tools:** ElectronJS, ReactJS, robotJS, Ollama
>
> **Links:** GitHub · Demo

ReNameify: Content-Aware PDF Batch Renamer DESKTOP APP (2024)

Written for one real user, not a spec. My sister was renaming exported PDFs by hand (opening each one, reading the header, retyping it), so I sat with her, watched the actual workflow, and shipped a one-click Windows installer that does it in a folder-at-a-time pass. It extracts each PDF's text, then scores lines with a header-detection heuristic that rejects the things that look like titles but aren't: page numbers (`page 12`, `- 12 -`), sentences ending in a full stop, lines that are mostly punctuation, and anything outside a sane length band. The heuristic is Korean-aware, since half her documents were in Korean.

Trust mattered more than accuracy here. Nothing is renamed in place. Originals stay untouched and copies land in a `renamed_files/` subfolder, a PDF that fails to parse (scanned pages, no text layer) keeps its original name instead of erroring the batch, and a first-run guided tour walks a non-technical user through the three buttons. It survived contact with its one user, which is the only test that counted.

> **Role:** Solo.

> **Tools:** ElectronJS, Node.js, pdf-parse, fs-extra, Shepherd.js, Winston, electron-builder (NSIS one-click installer)
>
> **ML/Algorithms:** Rule-based header extraction from first-page text: length banding, terminal-punctuation and page-number rejection, special-character-ratio filtering with a Hangul-safe character class, document-specific anchor matching, and filename sanitization with graceful fallback to the original name
>
> **Links:** GitHub

RESEARCH AND APPLIED ML PROJECTS (2023–2024)

Where the model or the method was the deliverable.

FastRAG: Self-Hosted Document RAG Backend (2024)

No API key, no data leaving the machine. FastRAG is a document Q&A backend that comes up with a single `docker compose up`: a FastAPI parse/chat service, Qdrant for vectors, a CUDA llama.cpp server running a quantized Llama-3.2-3B, and a Gradio demo UI.

Ingestion offers two paths: a custom PyMuPDF reader with tunable chunk size and stride, plus an optional bridging pass that stitches the tail of one page to the head of the next so an answer split across a page break is still retrievable as one chunk; or layout-aware Markdown chunking for documents where structure carries the meaning.

Retrieval is bge-m3 dense search with a score floor, and the prompt is written to refuse rather than improvise when the retrieved references don't cover the question. Follow-ups can be rewritten into standalone queries against conversation history kept in Qdrant, and answers stream token-by-token from llama.cpp through FastAPI to the UI, with per-response latency logged and shown to the user.

> **Role:** Solo.
>
> **Tools:** Python, FastAPI, Qdrant, llama.cpp (CUDA server), LlamaIndex, PyMuPDF/pymupdf4llm, LangChain text-splitters, Gradio, Docker Compose, Poetry, pre-commit
>
> **ML/Algorithms:** BAAI/bge-m3 dense embeddings with cosine search and a score threshold; on-disk vectors and memmap-tuned Qdrant collections for low-RAM hosts; page-boundary overlap chunking vs. layout-aware Markdown chunking; history-conditioned query rewriting; grounded-answer prompt with an explicit refusal path
>
> **Links:** GitHub

Probabilistic Multiview Object-Pose Estimation (2023)

One fused pose distribution per object, rather than a best-view pick. Single-view estimators disagree with each other, and choosing between them throws information away. We associated multiple estimation

results with scene graphs, combined the pose distributions coming out of single-view estimators, and constructed a unified scene graph by predicting one pose distribution per object. On table-top object-pose scenarios the approach **beat three baseline methods** on both position and orientation accuracy.

> **Role:** Neural architecture research for pose estimation and scene graph generation, then training and combined efficient inference. My teammate handled environment setup, simulation dataset creation, and visualization.
>
> **Links:** Draft paper

6D Pose Estimation and Scene Graph Generation (2023)

Two heavy frameworks in one pipeline, and the cost shows up as framerate. I combined the Cosypose framework for 6D pose with Neural-Motif for scene graph generation and ran experiments on custom cup datasets, a single-camera setup producing both 6D pose and a dynamic scene graph. For 6 cups, with pose estimation plus relationship and attribute predictions, the combined framework ran at **1.25 frames/s**.

> **ML/Algorithms:** Cosypose: CNN with geometric optimization (ICP registration and non-linear optimization via Levenberg-Marquardt); Neural-Motif: Region Proposal Network with bi-directional LSTM
>
> **Links:** Demo

MBTI Prediction (2023)

One classifier pair per MBTI axis, not one model for all sixteen types. Submitted to the Qualcomm-KAIST Innovation Awards Hackathon in May 2023, predicting MBTI (a widely used personality assessment) from text. The model is a hierarchical classifier: 4 fine-tuned KoBERT classifiers and 4 SVM classifiers, one pair per section of the MBTI test.

For robustness I augmented the data by paraphrasing long answers with google-t5-base, and ran EDA on the question information to extract a new feature set from the questions themselves. Fine-tuning KoBERT meant adding a fully connected layer with softmax activation at the end, then adaptively fine-tuning by varying the amount of layer-freezing in two steps.

I placed **3rd on the Phase-2 private leaderboard**. The slides cover the results, the other approaches I tried, and future work.

> **ML/Algorithms:** KoBERT fine-tuning, SVM, hierarchical classification, paraphrase-based data augmentation
>
> **Links:** Presentation

ESL Fusion: Junction Asia Hackathon (2023)

Electronic shelf labels as the output device for a store recommender. Using SOLUM's ESL hardware, we built recommendation and stock tracking for offline stores on displays cheap and low-power enough to scale to every shelf. Stock data fed both a visual analysis layer for insights and a time-series model projecting the next day's sales.

> **Role:** Team lead and product. I ran brainstorming, assigned roles and built the presentation slides. On the technical side I worked out from SOLUM's API manuals how to POST our content to the ESL screens, and built a QR-code-accessible web dashboard so customers on the local network could see the visualizations.
>
> **Tools:** REST API, qrcode, Dash HTML, Matplotlib, Flask, Figma
>
> **Links:** GitHub

EARLIER PROJECTS AND COURSEWORK

University coursework, team projects and first builds.

B-bot (Personal Chatbot) (2023)

An early resume chatbot, pre-embeddings: an MLP classifier over a Bag of Words representation maps a question to one of a set of predefined tags and returns the matching answer, with stemming and spell-checking on the way in. Streamlit front end, hosted on Streamlit Cloud.

> **Tools:** Streamlit-chat, PyTorch, PySpellchecker, NLTK
>
> **ML/Algorithms:** Multi-Layer Perceptron classifier, Bag of Words representation
>
> **Links:** Live chatbot · GitHub

Paraphrase Generation (QCPG Replication)

We replicated *Quality Controlled Paraphrase Generation* (Bandel et al., ACL 2022).

> **Role:** Three improvements. I ablated the Quality Predictor to measure its actual contribution, swapped t5-base for multilingual mt5-base to train on Korean, and wrote the end-to-end inference pipeline from scratch.
>
> **Tools:** PyTorch, Matplotlib, transformers, Hugging Face models
>
> **Links:** Replication report · GitHub

AI-empowered F1/10 Autonomous Racing Car

Course team project. I set up the Jetson Xavier NX environment, networking and ROS sensor packages, and owned perception. Yolact_ros instance segmentation proved too computationally heavy, so obstacle avoidance moved to LIDAR and vision was cut back to traffic-light detection by a rule-based crop, mask and count-pixels classifier. AMCL and Hector Odom for localization, PID for control.

> **Tools:** Yolact_ros, OpenCV, Linux, ROS, NVIDIA SDK, Arduino IDE
>
> **Links:** Final report

Crypto Price Predictor

Seven forecasting models for the price of bitcoin (recurrent, convolutional, attention-based and a Bayesian network) across one-day and long-sequence horizons, the longest running to 100 days.

> **Role:** I built the two deep models for the one-day task, one recurrent (bidirectional LSTM) and one convolutional (Conv1D), and ran multivariate forecasting with both.
>
> **Tools:** TensorFlow, Pandas, Matplotlib
>
> **ML/Algorithms:** Conv1D, bi-directional LSTM
>
> **Links:** Project paper · GitHub

Movie Recommendation (Netflix Prize style)

Given `<USER ID>`, `<MOVIE ID>`, `<TIMESTAMP>` rows, I filled the missing `<RATING>` column by factorizing the sparse user-movie utility matrix as $R \approx UV^T$ and training both factors with Alternating Least Squares against RMSE. Two-fold normalization first (user mean, then movie mean), then denormalize to predict.

> **Tools:** NumPy, Python
>
> **ML/Algorithms:** Dimensionality reduction (UV-decomposition), Alternating Least Squares

---

Pintos

pintos-kaist, a simple x86-64 OS framework forked from Stanford's Pintos and run under QEMU, for my Operating Systems course. Across four projects: thread synchronization with interrupts (semaphores, locks,

condition variables), a system call handler for I/O in user programs, virtual memory with page-fault handling and swapping, and the basic file system.

> **Links:** Project reference

Smart Home

A security tech project detecting intruders and notifying the house owner.

> **Role:** Hardware setup (connecting the sensor to a Raspberry Pi via breadboard) and the Node-RED workflow, connecting each process to a Telegram bot for automated response.
>
> **Tools:** Node-RED
>
> **Links:** Demo video · GitHub

Data Visualization Task

A visualization task given to me by a startup before hiring me as an intern in Winter 2022.

> **Tools:** D3.js, Colab (data preprocessing), Flask, Heroku
>
> **Links:** Live app · GitHub